ESAIP · MASTER 1 MSc CYBERSECURITY AND DATA SCIENCE · PROJET APPLICATIF 3

SENTINEL-FL

Federated Learning for Cross-Border Financial Crime Detection

Final report · Phases 1 to 6 · 9 November 2026

Team	Vijay Dipak Bhosale — Project Manager, Regulatory & Risk Lead Abdul Fathah Chappan Puthiya Purayil — Federated Learning & AI Lead Mereena Mariam Jibu — Cybersecurity & Privacy Lead Niranjan Satheesh Thandassery — System Architecture & Auditability Lead
Referent	Sofiane Hamrioui
Project period	22 September – 10 November 2026 (8 weeks)
Defence	Tuesday 10 November 2026, with live demonstration
Deliverables	This report · prototype repository (sentinel-fl) · live demonstration · defence slides · conference proposal

Abstract

European banks must detect money laundering that crosses institutions and borders, yet GDPR, banking secrecy and the AML tipping-off rule prevent them from pooling transaction data. We designed, built and evaluated **SENTINEL-FL**, a federated learning consortium in which three simulated banks in France, Germany and the Netherlands train one detection model without any transaction leaving its bank. Every update is signed, checked against a smart-contract participant registry, screened for poisoning, aggregated robustly and recorded in a signed hash-chain ledger whose per-round Merkle roots are anchored in the contract. Differential privacy (DP-SGD with Rényi accounting) bounds what an update can reveal. The system runs as separate bank and aggregator nodes with 14 web pages for bank staff, the operator and the regulator, on a reference NumPy engine and on Flower + PyTorch + Opacus.

On synthetic AMLworld-like data (60,000 transactions, 1% illicit), federation reaches PR-AUC **0.859 ± 0.028** over three seeds, against 0.840 for a centralised model and 0.715 for banks working alone. A label-flipping bank drives an undefended model to 0.012; with our defences the model stays at 0.835 and the attacker is quarantined in its second malicious round. Backdoor success falls from 1.0 to 0.0, exfiltration and tampered updates are blocked by the protocol, an insider edit to the ledger is located at the exact record, and an incident is reconstructed from the ledger and the on-chain anchors in about a millisecond. Privacy has a measurable price: PR-AUC 0.703 at $\varepsilon \approx 6$ and 0.566 at $\varepsilon \approx 3$.

Contents

Abstract . . . 1

Contents **2**

1. Introduction **4**

1.1 Context . . . 4

1.2 Problem statement . . . 4

1.3 Contributions . . . 4

1.4 Method and organisation . . . 4

2. Phase 1 — Scoping and problem definition **6**

2.1 Scope and assumptions . . . 6

2.2 Assets . . . 6

2.3 Threat model . . . 6

2.4 Regulatory constraints . . . 7

2.5 Requirements derived . . . 7

3. Phase 2 — State of the art and critical analysis **8**

3.1 Federated learning and frameworks . . . 8

3.2 Privacy-preserving techniques . . . 8

3.3 Robust aggregation and attacks . . . 9

3.4 Blockchain-based federated learning . . . 9

3.5 Financial crime detection . . . 9

3.6 Selected technologies . . . 9

4. Phase 3 — Solution design **11**

4.1 Architecture . . . 11

4.2 Round protocol . . . 11

4.3 Privacy design . . . 12

4.4 Auditability and governance . . . 12

4.5 Attack scenarios . . . 12

5. Phase 4 — Implementation **14**

5.1 Repository . . . 14

5.2 Two engines, one security layer . . . 14

5.3 Data and model . . . 14

5.4 Distributed system and operational pages . . . 15

5.5 Smart-contract registry . . . 16

5.6 Alerts, copilot and monitoring . . . 16

5.7 Test environment . . . 16

5.8 Experimentation journal (main entries) . . . 17

6. Phase 5 — Testing and evaluation **18**

6.1 Protocol . . . 18

6.2 Federated versus centralised (scenario 5) . . . 18

6.3 Malicious participant (scenario 1) . . . 18
6.4 Privacy budget consumption (scenario 2) . . . 19
6.5 Leakage (scenario 3) . . . 20
6.6 Audit reconstruction after an incident (scenario 4) . . . 20
6.7 Overhead . . . 21
7. Critical analysis **22**
7.1 What the results show . . . 22
7.2 Performance and data . . . 22
7.3 Security . . . 22
7.4 Privacy and regulation . . . 22
7.5 Auditability . . . 22
7.6 Engineering limits . . . 23
7.7 Improvement paths . . . 23
8. Conclusion **23**
Annex A. Screenshots of the running system **24**
Annex B. Traceability to the brief **30**
Annex C. Team contributions **30**
Annex D. Reproducing the results **30**
References **31**

1. Introduction

1.1 Context

Money laundering is a network problem. A typical scheme splits funds into many transfers just below reporting thresholds, routes them through accounts at several banks, often in several countries, and recombines them. Each bank sees only its own fragment of the pattern. Detection would improve if banks could learn from each other's labelled cases, but three constraints stand in the way: the GDPR limits processing and transfer of customers' personal data, banking secrecy restricts disclosure of client information, and the anti-money-laundering (AML) framework forbids disclosing that a suspicion report exists (the tipping-off prohibition).

Federated learning (FL) changes what is shared: banks keep their data and exchange only model updates. The brief for this project cites MedLedgerFL, which combines blockchain and FL so that model updates are auditable and only authorised institutions take part, and asks us to transfer those principles to financial crime, where data is heterogeneous, regulation is fragmented and alerts are needed in near real time.

1.2 Problem statement

How can a consortium of European banks collaboratively train a financial crime detection model using federated learning, while providing cryptographic proof that no transaction data left its jurisdiction and that the model was not compromised?

The question has three parts that we treat as separate requirements: **detection quality** (the shared model must be worth having), **sovereignty and privacy** (raw data must stay put, and updates must not leak it), and **integrity and accountability** (a supervisor must be able to verify, after the fact, who contributed what and that nobody poisoned or rewrote the record).

1.3 Contributions

- A regulatory and threat analysis that maps GDPR, AML and DORA obligations to concrete technical controls (sections 2 and 4).
- A state of the art of FL frameworks, privacy techniques, blockchain-based FL and AML detection, with a justified selection (section 3).
- An architecture with three trust layers: cryptographic participation (signed updates, smart-contract registry), statistical robustness (anomaly detection, quarantine, robust aggregation) and accountability (signed ledger, on-chain anchoring, regulator view) (section 4).
- A working prototype, SENTINEL-FL: separate bank and aggregator nodes with 14 operational pages, a ConsortiumRegistry smart contract, e-mail alerting, an audit copilot, Docker deployment and monitoring, on two engines (reference NumPy, and Flower + PyTorch + Opacus) (section 5).
- An evaluation of the five test scenarios of the brief, with seeds, overhead and a membership-inference test, followed by a critical analysis (sections 6 and 7).

1.4 Method and organisation

The project followed the six phases of the brief over eight weeks, planned in MS Project with continuous risk management and reported weekly to the referent. Roles followed the team work plan: the project manager also led regulation and risk; the three other members led federated learning, cybersecurity and privacy, and architecture and auditability. Annex C lists individual contributions.

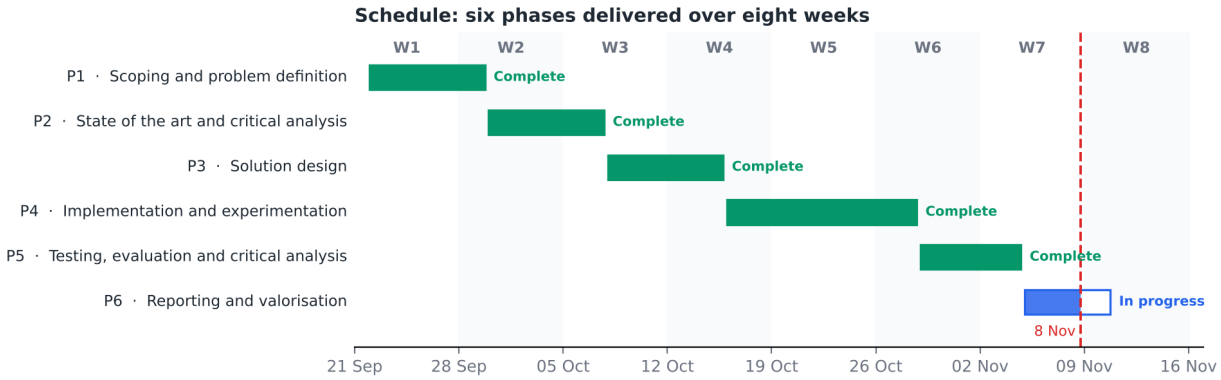


Figure 1. Project schedule (from the MS Project plan) and status on 8 November 2026.

2. Phase 1 — Scoping and problem definition

2.1 Scope and assumptions

- **Consortium:** three banks (A Paris, B Frankfurt, C Amsterdam) in three EU jurisdictions, scalable to five; one consortium operator running the aggregator; one supervisor (regulator) with read access and the power to suspend a participant.
- **Task:** binary classification of individual transactions as laundering or not, with a model shared by all banks and used locally for alerting.
- **Data:** no real customer data. We use a synthetic generator modelled on IBM AMLworld (fan-out, fan-in and cross-bank cycles, threshold structuring, night activity, currency mismatches), and a loader for the public AMLworld CSV files.
- **Adversary:** up to one malicious bank out of three (honest majority), a network attacker between a bank and the aggregator, and a malicious insider with write access to the audit database. The aggregator operator is trusted to run the protocol but is audited.
- **Out of scope:** case management and suspicious activity report (SAR) filing, real-time payment interception, and production key management.

2.2 Assets

Asset	Owner / location	Sensitivity	Protection goal
Transaction records (amounts, accounts, timestamps)	Each bank, never leaves it	Personal data, banking secrecy	Confidentiality, residency
Labels and alert cases (suspicions)	Each bank's AML unit	Tipping-off protected	Confidentiality; only counts may leave
Customer identities and account IDs	Each bank	Personal data	Never processed outside the bank
Model updates (weights, n, ε)	Bank → aggregator	May encode personal data	Integrity, authenticity, bounded leakage (DP)
Global model	Consortium	Shared intellectual asset	Integrity, traceability to its inputs
Participant keys and registry	Banks, contract	Security-critical	Authenticity, access control
Audit ledger and anchors	Operator, contract	Evidence for supervisors	Integrity, non-repudiation, availability

Table 1. Asset inventory.

2.3 Threat model

ID	Threat	Actor	Impact	Brief scenario
T1	Raw data or labels leave a bank (directly or hidden in an update)	Malicious or careless bank	GDPR breach, tipping-off	Cross-border leakage
T2	Reconstruction or membership inference from updates or the model	Aggregator, another bank, outsider	Privacy breach	Privacy budget
T3	Data poisoning (label flipping) or model poisoning (scaling, backdoor)	Malicious bank	Useless or biased model	Malicious participant
T4	Free-riding: benefit without contributing	Bank	Unfair, degraded model	—
T5	Update altered or forged in transit	Network attacker	Silent poisoning	—
T6	Impersonation of a bank or unauthorised node	Outsider	Poisoning, data access	Access control

ID	Threat	Actor	Impact	Brief scenario
T7	Rewriting the audit history after an incident	Insider	Loss of accountability	Audit reconstruction
T8	Regulatory violation by design (excess data, no record)	Consortium	Sanctions, loss of trust	—

Table 2. Threats considered. Section 4.5 maps each threat to controls and attack scenarios A1–A8.

2.4 Regulatory constraints

Instrument	Obligation relevant to the design	Design consequence
GDPR (EU) 2016/679	Data minimisation (Art. 5(1)(c)), protection by design (Art. 25), security of processing (Art. 32), impact assessment for high-risk processing (Art. 35), accountability (Art. 5(2))	Only weights, sample count, ε and alert counts leave a bank; allow-list enforced; DP; signed records
AML framework: Directive (EU) 2015/849 and Regulation (EU) 2024/1624 (AMLR), AMLA Regulation (EU) 2024/1620	Customer due diligence and transaction monitoring; prohibition of disclosure of suspicion reports (tipping-off, Art. 39 of 2015/849); the AMLR adds a framework for information-sharing partnerships	Case details stay in the bank portal; the consortium sees counts and pattern names only
Banking secrecy (national law)	Client information may not be disclosed to third parties	No customer attribute crosses a bank boundary
DORA (EU) 2022/2554	ICT risk management, incident logging and reporting, third-party risk	Signed ledger, incident timeline, alerts, monitoring
AI Act (EU) 2024/1689	Annex III lists creditworthiness assessment as high-risk but excludes systems used to detect financial fraud	Good practice: logging, human oversight (analyst confirms alerts), documentation

Table 3. Regulatory constraints and their design consequences. We are not lawyers; the table records engineering assumptions to validate with compliance experts.

An important nuance is that model updates are not automatically anonymous. Research on gradient inversion and membership inference [16, 17] shows that updates can reveal training data, so the consortium should treat them as potentially personal data and protect them accordingly; this is the regulatory argument for differential privacy rather than relying on 'we only share weights'.

2.5 Requirements derived

ID	Requirement	Verified by
R1	No raw transaction, label or identifier leaves its bank	Allow-list, residency log (test E9, residency page)
R2	Only registered, approved banks with the right key can contribute	Smart-contract registry, signatures (tests, T6)
R3	A poisoned contribution must not degrade the model; the attacker must be isolated	E6–E8, live attack
R4	Each bank's privacy loss is quantified and capped	RDP accountant, ε cap (E4, T2)
R5	Every round can be reconstructed and any rewrite of history detected	Ledger, Merkle roots, anchors (T5)
R6	The federated model is at least as good as each bank alone, close to centralised	E1–E5, seeds (T1)
R7	Supervisors get evidence without seeing data	Regulator page, ARIA, traceability matrix
R8	The overhead of the controls is acceptable	T4

Table 4. Requirements and how they are verified.

3. Phase 2 — State of the art and critical analysis

3.1 Federated learning and frameworks

Federated averaging (FedAvg) [1] trains a model by repeatedly sending the global weights to clients, letting each run a few local optimisation steps and averaging the results weighted by sample count. With non-IID data, as between banks of different size and clientele, client models drift apart; FedProx [2] adds a proximal term that keeps local models close to the global one. Kairouz et al. [3] survey the open problems, among them robustness, privacy and fairness, which this project addresses for AML.

Criterion	Flower [4]	FedML [5]	NVIDIA FLARE [6]	TensorFlow Federated
Framework-agnostic (PyTorch, NumPy)	Yes	Yes	Yes	TensorFlow only
Custom aggregation strategy	Easy: subclass a Strategy	Yes	Yes (controllers)	Yes, functional API
Simulation and real deployment with same code	Yes (simulation, SuperLink / SuperNode)	Yes	Yes (POC, provisioning)	Mostly simulation
Deployment weight for a student prototype	Light, pip install	Medium, MLOps platform	Heavy, enterprise provisioning	Medium
Security features	TLS, node authentication	Varies	Strong: PKI, privacy filters	Limited
Fit for this project	**Selected**, named in the brief	Good, heavier	Strong for production banks	Poor (framework lock-in)

Table 5. Comparison of FL frameworks.

Justification. Flower lets us write the defence pipeline as a custom strategy, run the same apps in simulation and on separate machines, and keep PyTorch and Opacus unchanged. NVIDIA FLARE offers stronger provisioning for production but would have consumed the schedule. We also kept a framework-free reference engine so that every security mechanism could be inspected and tested without the framework (section 5.2).

3.2 Privacy-preserving techniques

Technique	Protects against	Cost	Limits	Decision
Differential privacy, DP-SGD [7] with Rényi accounting [8], Opacus [9]	Membership inference, reconstruction from the update or the model	Accuracy loss; per-sample gradients	Utility drops quickly for rare classes	**Used** per bank, ε capped
Secure aggregation [10]	Aggregator seeing individual updates	Extra rounds of masking, dropout handling	Server can no longer inspect updates → conflicts with poisoning detection	Not used; discussed (7.3)
Homomorphic encryption [11]	Aggregator seeing updates	10^3–10^6× compute, large ciphertexts	Same conflict; heavy	Not used
Trusted execution environments	Operator reading updates	Special hardware, attestation	Side channels, vendor trust	Improvement path
Data minimisation by protocol (allow-list)	Data smuggled in updates	Negligible	Only covers declared fields	**Used**

Table 6. Privacy techniques compared.

A central tension appears here. Secure aggregation and homomorphic encryption hide individual updates from the aggregator, but robust aggregation and anomaly detection need to see them. With only three banks, an attacker hidden inside an encrypted sum cannot be singled out. We therefore chose **visible but privatised** updates: DP limits what each update can reveal, and the aggregator can still inspect, exclude and quarantine.

3.3 Robust aggregation and attacks

Byzantine-robust rules replace the mean: Krum and Multi-Krum select updates close to their neighbours [12]; the coordinate-wise median and trimmed mean discard extremes [13]. Attacks have kept pace: boosted model replacement implants backdoors [14], and optimised local model poisoning can defeat several robust rules [15]. Robust rules alone also treat symptoms: they down-weight an attacker without identifying it. For accountability, a consortium needs detection and a decision (quarantine), recorded with evidence, which is why we combine scoring, quarantine and a ledger with a robust rule.

3.4 Blockchain-based federated learning

Blockchain FL designs (for example BlockFL [20], and MedLedgerFL as cited in the brief) record updates or their hashes on a ledger, reward or authorise participants through smart contracts, and sometimes replace the central aggregator with consensus. Permissioned platforms such as Hyperledger Fabric [26] suit consortia of known institutions. Their limits are well documented: model weights do not belong on-chain (size, cost, and they may be personal data), consensus adds latency, and a ledger proves only what was written to it, not that it was true. We therefore keep weights off-chain, write signed hashes to an operator ledger, and use a smart contract for what blockchains are good at: shared rules for membership and write-once anchoring of each round's Merkle root.

3.5 Financial crime detection

Rule-based monitoring generates many false positives. Supervised models on transaction features, and graph neural networks on transaction graphs [19], improve detection; the AMLworld datasets [18] provide realistic synthetic transactions with laundering patterns because real labelled data cannot be published. Federated approaches for financial crime have been proposed, for example federated graph learning across institutions [28]. Laundering is rare (around 0.1–1% of transactions), so accuracy is meaningless; we report PR-AUC, F1 and recall at a 1% false-positive rate.

3.6 Selected technologies

Need	Choice	Reason
Orchestration	Flower 1.18 (ServerApp, ClientApp, SuperLink/SuperNode) + reference engine	Named in brief; custom strategy; simulation and deployment
Model	PyTorch MLP 25 → 32 → 16 → 1	Small enough for DP-SGD, tabular features
Differential privacy	Opacus PrivacyEngine (RDP), own accountant as cross-check	Named in brief; independent verification of ε
Robustness	Trimmed mean / Multi-Krum + anomaly scores + quarantine + norm clipping	Identifies the attacker, not only dampens it
Identity and integrity	Ed25519 signatures	Fast, small keys, standard
Access control	ConsortiumRegistry smart contract (Solidity, local Ethereum chain or simulated)	Shared, enforceable membership rules
Audit	Signed SQLite hash chain + Merkle roots anchored in the contract	Tamper evidence without putting data on-chain

Need	Choice	Reason
Deployment and monitoring	Docker Compose, Prometheus, Grafana	Named in brief; one command

Table 7. Technology choices.

4. Phase 3 — Solution design

4.1 Architecture

The consortium has three kinds of node. **Bank nodes** hold their data, train locally, score their live transaction stream and sign their updates. The **aggregator** orchestrates rounds and runs the defence pipeline. The **registry contract** holds membership and access rules and the round anchors. A **regulator** reads evidence and may suspend a bank on the contract. Banks only make outbound calls, like Flower clients, so no bank needs to open a port.

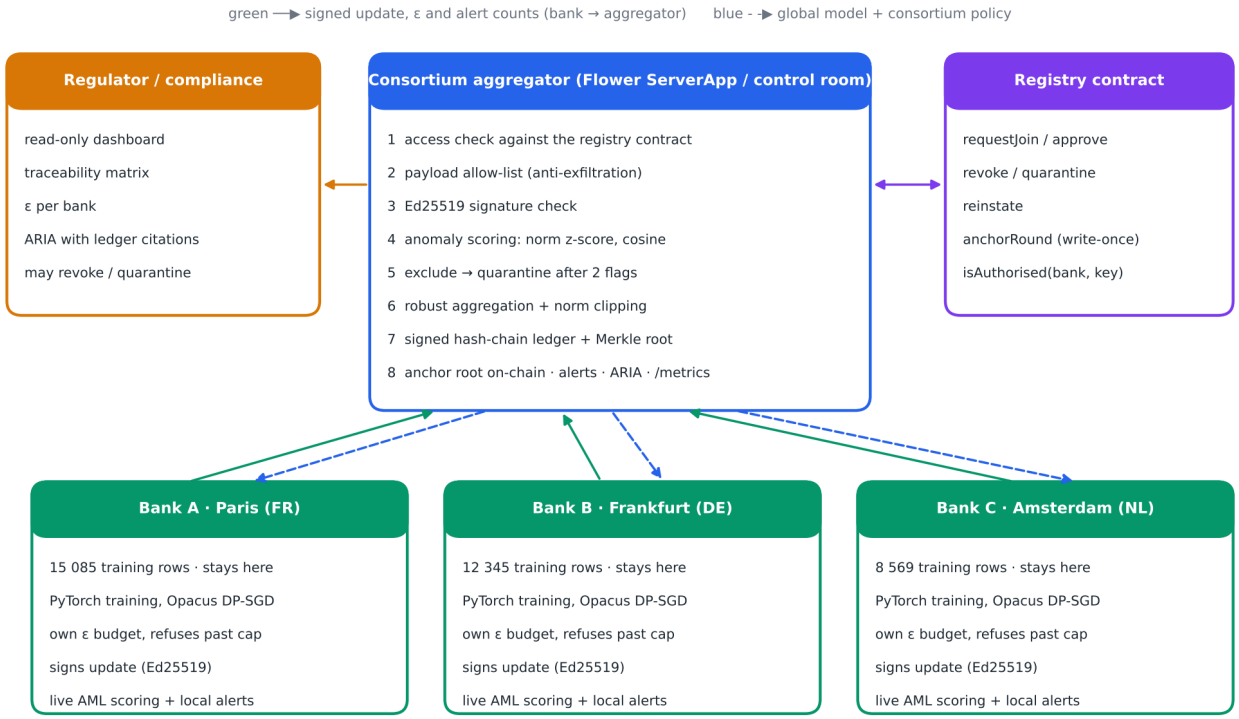


Figure 2. SENTINEL-FL architecture. Data stays in the green boxes; only signed metadata crosses the arrows.

4.2 Round protocol

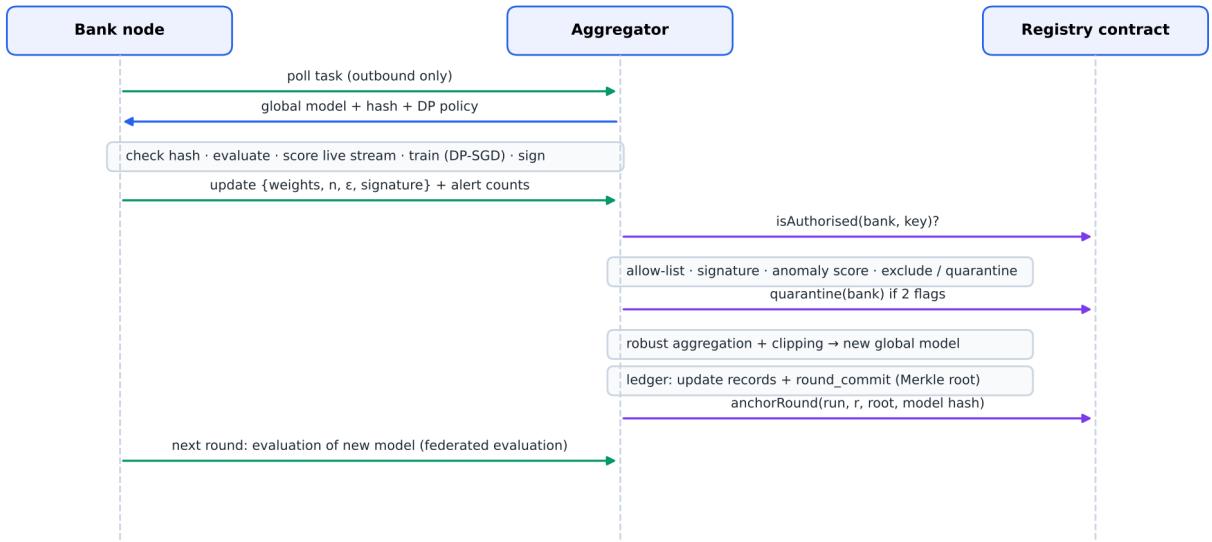


Figure 3. One federated round. Evaluation of a model version happens at the start of the next round, on each bank's own test data.

The aggregator publishes the global model with its hash and the consortium policy (rounds, aggregation rule, DP parameters). Each bank checks the hash, evaluates the model on its local test data, scores its next slice of live transactions, trains for a fixed number of steps, and returns the new weights with its sample count, privacy spend and an Ed25519 signature over *(bank, round, hash(weights), n, ε)*. The aggregator applies, in order: the contract check, the payload allow-list, the signature check, anomaly scoring, exclusion and quarantine, robust aggregation with server-side norm clipping, ledger recording and on-chain anchoring.

4.3 Privacy design

- **Data minimisation by protocol.** The update payload is an allow-list (bank, round, weights, n, ε, signature, status). Any extra field is treated as an exfiltration attempt: the update is discarded, the event logged and the bank quarantined.
- **Differential privacy at the bank.** DP-SGD clips each example's gradient to norm C and adds Gaussian noise σC to the sum, with Poisson sampling. Each bank accounts its own ε with a Rényi-DP accountant ($\delta = 10^{-5}$) and refuses to train once its next round would exceed the consortium cap; the consortium policy fixes σ, C and the cap, so all banks give the same guarantee.
- **Alert minimisation.** Each bank scores live transactions and keeps case details inside its portal; only counts and pattern names reach the consortium, respecting the tipping-off prohibition.
- **Residency evidence.** Every outbound message is logged by the bank with its size and field names, so a bank can show a supervisor that no raw row left.

4.4 Auditability and governance

The audit ledger is an append-only SQLite table in which each record contains the hash of the previous record and is signed by the operator's Ed25519 key. For each round, the Merkle root of the accepted update hashes is committed to the ledger and anchored in the contract with `anchorRound`, which is write-once. A verifier recomputes every hash and signature and reports the first inconsistent record; an auditor can recompute each round's root from the update records and compare it with the anchor.

Role	May	May not
Bank	Request to join with its key; train; see its own data, alerts and residency log	Approve itself; see other banks' data or cases
Operator (aggregator)	Approve, revoke, quarantine, reinstate; run rounds; anchor roots	Rewrite anchored roots; read bank data
Regulator	Read all evidence; revoke or quarantine a bank on the contract; query ARIA	Approve a bank (separation of duties)

Table 8. Governance roles enforced by the registry contract.

4.5 Attack scenarios

ID	Scenario	Threat	Expected control	Test
A1	Label flipping by Bank C	T3	Anomaly score → exclusion → quarantine; robust rule	E6, live demo
A2	Backdoor with boosted update (trigger: currency mismatch)	T3	Norm z-score, server clipping	E7
A3	Boosted scaling of an update	T3	Norm z-score, clipping	Attack lab
A4	Reconstruction / membership inference	T2	DP-SGD, ε cap	E4, T3

ID	Scenario	Threat	Expected control	Test
A5	Raw-data exfiltration inside the update	T1	Allow-list, quarantine, residency log	E9
A6	Unauthorised node / impersonation	T6	Contract isAuthorised, key pinning	Unit tests
A7	Update altered in transit	T5	Signature check	E10
A8	Free-rider returns the global model	T4	Cosine and norm scores	E8

Table 9. Attack scenarios A1–A8 and the controls expected to stop them.

5. Phase 4 — Implementation

5.1 Repository

The prototype is a single Git repository (sentinel-fl) with 21 automated tests. Its main parts:

Path	Content
sentinel/	Reference engine: data generator and loader, NumPy MLP with per-sample gradients, DP-SGD and RDP accountant, aggregation rules, attacks, crypto, ledger
sentinel/aggregator_node.py, bank_node.py	Distributed system: aggregator and bank nodes over HTTP, attack lab, alerts, e-mail, ARIA, Prometheus metrics
sentinel/registry_chain.py, contracts/	ConsortiumRegistry contract (Solidity), simulated and Ethereum backends, deploy script
sentinel_flower/	Flower ServerApp (SentinelStrategy) and ClientApp, PyTorch model, Opacus training, framework-free RoundGuard
web/	8 aggregator pages and 6 bank-portal pages (HTML/JS, no external dependency)
configs/, experiments/, scripts/	Experiment configurations, results (matrix, Phase 5), measurement scripts
docker-compose*.yml, Dockerfile*, monitoring/	Containers per node, Flower and chain profiles, Prometheus and Grafana
docs/	Architecture, one-laptop demo, multi-laptop setup, operations guide, Flower, smart contract, alerts
tests/	Core, distributed nodes over HTTP, contract rules, Flower glue (21 tests)

Table 10. Repository structure.

5.2 Two engines, one security layer

The **reference engine** implements the model, per-sample gradients, DP-SGD and the RDP accountant in NumPy, so every mechanism can be read and unit-tested. It powers the experiment matrix and the distributed demo. The **Flower engine** (sentinel_flower) uses Flower's ServerApp and ClientApp, a PyTorch model of the same architecture, and Opacus' PrivacyEngine for DP-SGD; each client also computes ε with our own accountant as a cross-check. Both engines call the same security layer: the server-side pipeline is written as a framework-free `RoundGuard` that the Flower strategy wraps, and both write the same ledger and contract records, so the same verifier, dashboard and ARIA work on either.

5.3 Data and model

The generator produces 60,000 transactions over 20 days with 1% laundering. Banks differ in size, illicit rate and behaviour, so the partition is non-IID by construction (Figure 4). Each bank splits its own rows by time into 60% training, 20% validation and 20% test, and computes 25 features on its own rows only: amounts, currency mismatch, cross-bank flag, time of day, threshold proximity, per-account daily velocity, payment format and currency. The model is an MLP with 1,377 parameters trained with a positive-class weight to counter imbalance.

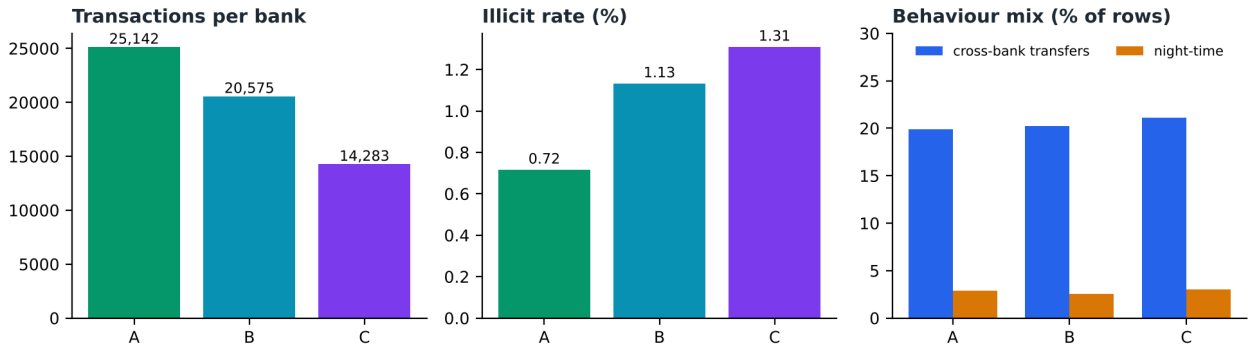


Figure 4. Non-IID partition: banks differ in size, illicit rate and behaviour.

5.4 Distributed system and operational pages

Each bank runs as its own process with its own keys, data, logs and web portal; the aggregator runs the control room. On defence day all nodes run on one laptop (`python -m sentinel showcase`); the same nodes also run on four laptops over a hotspot, or in Docker. The 14 pages expose every operation (Table 11); screenshots are in Annex A.

Node	Pages
Aggregator	Control room (map, run controls, update checks, global PR-AUC, events) · Registry & access (approve, revoke, reinstate; contract state and transactions) · Training rounds · Attack lab · Audit ledger (verify, insider-edit test) · Alerts & e-mail · Regulator · ARIA · Results
Each bank	Overview · Transactions (private live stream with scores) · AML alerts · Local training (signed update hash, bytes sent) · Privacy budget (ε vs cap) · Data residency (every outbound message, 0 raw rows)

Table 11. Operational pages.

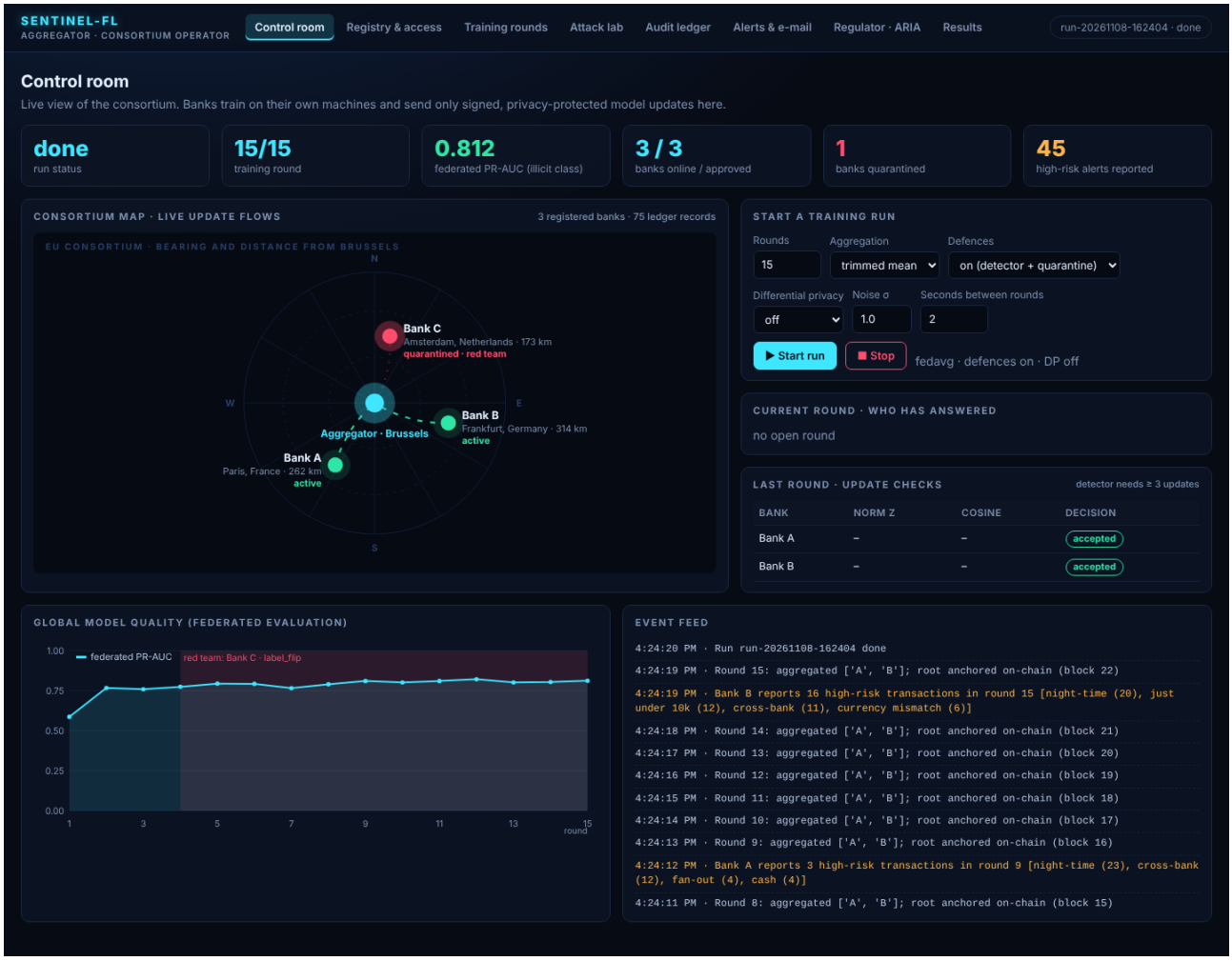


Figure 5. Aggregator control room after a live attack run: Bank C quarantined, PR-AUC stable, every round anchored on-chain (event feed).

5.5 Smart-contract registry

ConsortiumRegistry (Solidity 0.8) stores, per bank, the Ed25519 public key, jurisdiction, wallet and status (pending, approved, revoked, quarantined) and, per run and round, the Merkle root and model hash. The operator approves, quarantines and reinstates; the regulator may revoke or quarantine but not approve; anchors are write-once. The aggregator accepts an update only if `isAuthorised(bank, key)` is true. Two backends implement the same rules: a deterministic simulated chain with hash-linked blocks (default, used in tests and the demo) and a web3 client for the contract deployed on a local Ethereum chain (Anvil).

5.6 Alerts, copilot and monitoring

Banks raise high and medium alerts per transaction, tagged with patterns (just under 10k, night-time, cross-bank, currency mismatch, fan-out). Serious events (quarantine, exfiltration attempt, forged update, ledger failure, bursts of high-risk transactions) are pushed live to the dashboards and sent by e-mail over SMTP, or written to an outbox when SMTP is not configured. ARIA answers supervisors' questions by calling read-only tools on the ledger and state, cites record numbers and drafts incident reports, using a local LLM via Ollama or a deterministic offline mode. The aggregator exports Prometheus metrics with a provisioned Grafana dashboard.

5.7 Test environment

Item	Setting
Hardware	Development: 2-core cloud container; demo: Windows laptop with WSL 2 and Docker Desktop

Item	Setting
Software	Python 3.11–3.12, NumPy, pandas, cryptography; Flower 1.18, PyTorch 2.5 (CPU), Opacus 1.5.2; Docker Compose; Prometheus, Grafana; Anvil
Topologies	(1) one laptop, separate processes; (2) Docker Compose, one container per node; (3) four laptops over a phone hotspot
Data	Synthetic AMLworld-like, seed 42 (and 7, 2026 for variance); AMLworld CSV loader available
Training	20 rounds (matrix) or 15 (live), 40 local steps, batch 256, lr 0.05, momentum 0.9; DP runs: clip 5, lr 0.3, $\delta = 10^{-5}$
Defences	Warm-up 2 rounds, $z > 3.5$ or cosine $< 0 \rightarrow$ exclusion, quarantine after 2 flags, trimmed mean (trim 1), clip at 1.5× median norm

Table 12. Test environment.

5.8 Experimentation journal (main entries)

Date	Observation	Decision
16 Oct	DP collapsed PR-AUC to 0.07: per-sample clipping at C = 1 cancelled the positive-class weight	Clip 5 and lr 0.3 for DP runs
16 Oct	Honest Bank B flagged in round 2: early updates are naturally diverse	Warm-up of 2 rounds before anomaly scoring
16 Oct	Tamper demo changed a field to the same value	Edit flips booleans or writes a marker
16 Oct	Single-page dashboard insufficient for operations	Version 2: separate nodes and 14 pages
27 Oct	All high-risk alerts looked equal	High and medium tiers relative to each bank's operating threshold
8 Nov	Membership-inference test compared old training rows with newer validation rows (time shift)	Random in/out split within the training period
8 Nov	Flower engine and contract added; Flower runtime not installable in the build container	Glue tests with stand-ins; validation run on the team laptop

Table 13. Selected journal entries; the full history is in the Git log and the weekly reports.

6. Phase 5 — Testing and evaluation

6.1 Protocol

Unless stated otherwise, results come from the reference engine on the synthetic dataset (seed 42) with 20 rounds; the live-attack and audit results come from the distributed system (four processes over HTTP, 15 rounds). The main metric is PR-AUC on the illicit class over the union of the banks' test sets (12,001 transactions, 130 illicit). All numbers are reproducible with `python -m sentinel matrix`, `scripts/phase5.py` and `scripts/phase5_mia.py`.

6.2 Federated versus centralised (scenario 5)

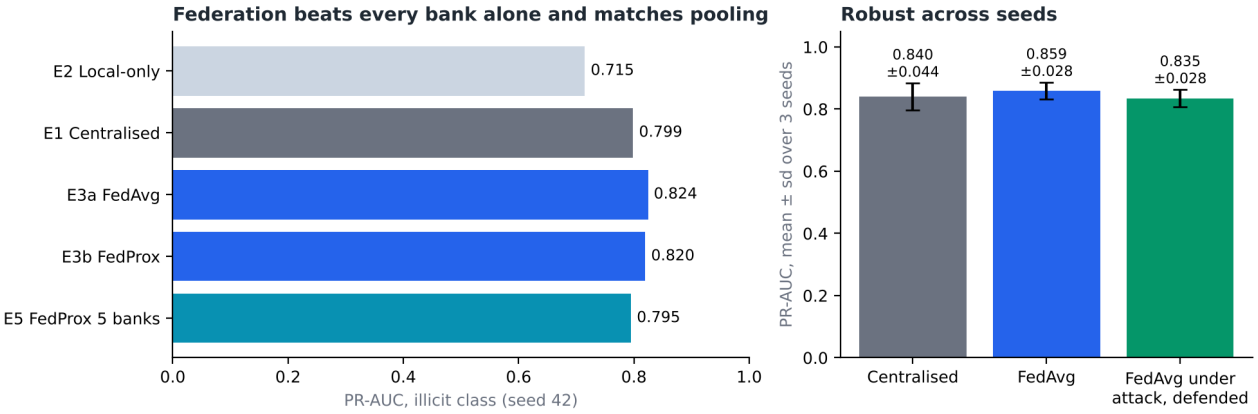


Figure 6. Detection quality: local-only, centralised and federated (left, seed 42); mean ± standard deviation over three seeds (right).

Federation improves every bank: the mean local-only PR-AUC is 0.715 against 0.824 for FedAvg. Over three seeds FedAvg reaches 0.859 ± 0.028 and the centralised model 0.840 ± 0.044, so the federated model is statistically indistinguishable from pooling the data, without pooling it. FedProx brings no gain here (the proximal term matters more with stronger heterogeneity), and five banks give 0.795 because the extra banks are small. At the best-F1 threshold the federated model has precision 0.76 and recall 0.75 (98 of 130 laundering transactions found, 31 false alarms among 12,001 transactions).

6.3 Malicious participant (scenario 1)

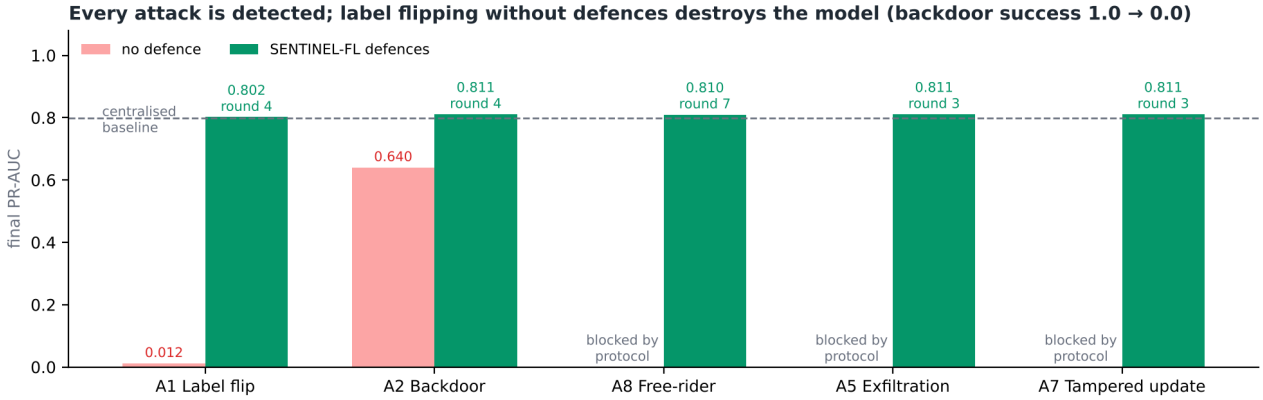


Figure 7. Final PR-AUC under attack, without and with defences (experiment matrix, 20 rounds).

ID	Experiment	PR-AUC	Backdoor success	Attacker caught
E6a	Label flip by Bank C, FedAvg, no defence	0.012	–	not caught
E6b	Label flip, trimmed mean + anomaly detector	0.802	–	Bank C in round 4

ID	Experiment	PR-AUC	Backdoor success	Attacker caught
E6c	Label flip, 5 banks, Multi-Krum	0.764	–	not caught
E7a	Backdoor (boosted) by Bank C, no defence	0.640	1.0	not caught
E7b	Backdoor, anomaly detector + server norm clipping	0.811	0.0	Bank C in round 4
E8	Free-rider, anomaly detector	0.810	–	Bank C in round 7
E9	Raw-data exfiltration attempt	0.811	–	Bank C in round 3
E10	Update altered in transit (MITM)	0.811	–	Bank C in round 3

Table 14. Attack experiments. The attack starts in round 3 or 4; detection is effective from round 3 (after warm-up).

Without defences, label flipping by one bank out of three destroys the model (PR-AUC 0.012) and the boosted backdoor succeeds on every triggered transaction. With the anomaly detector and robust aggregation, Bank C is excluded in the first round its behaviour changes and quarantined in the second; the model ends at 0.802, and over three seeds 0.835 ± 0.028, the honest level. Backdoor success drops to 0.0. A free-rider is caught later (round 7) because returning the global model only looks anomalous once honest updates converge. Multi-Krum with five banks (E6c) limits the damage (0.764) without identifying the attacker, which illustrates why we pair a robust rule with explicit detection.

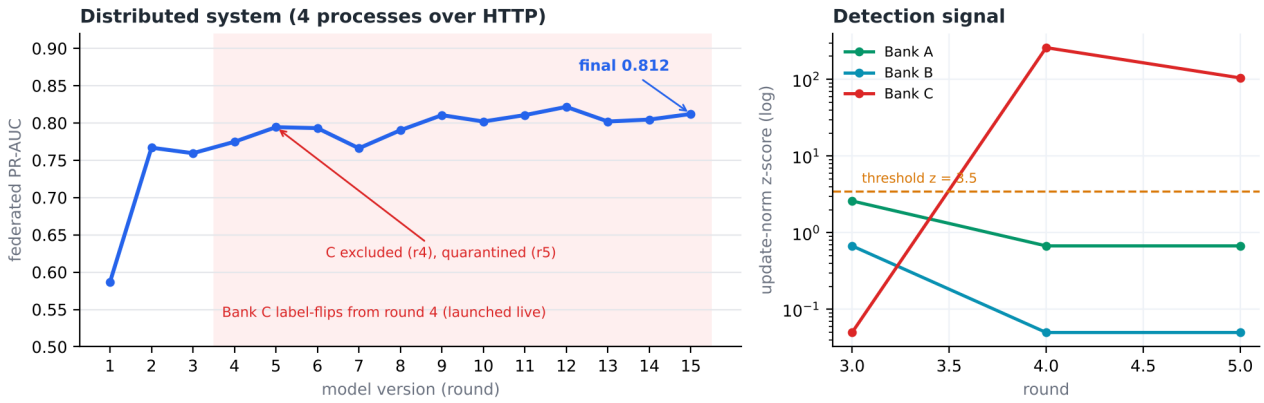


Figure 8. Live attack on the distributed system: Bank C's update norm jumps to z = 260; it is excluded in round 4, quarantined in round 5, and the model ends at 0.812.

6.4 Privacy budget consumption (scenario 2)

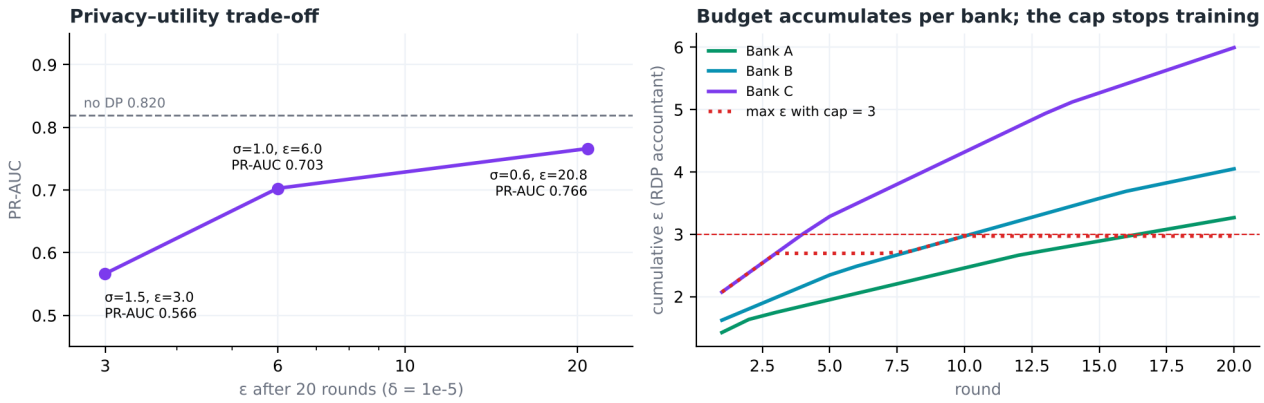


Figure 9. Left: utility against privacy (noise multipliers 1.5, 1.0, 0.6). Right: cumulative ε per bank (σ = 1.0) and enforcement of a cap of 3.

Privacy has a clear price. At ε ≈ 20.8 the model keeps most of its quality (0.766); at ε ≈ 6, a commonly used operating point, it reaches 0.703; at ε ≈ 3 it falls to 0.566. Budgets grow at different speeds because sampling rates differ: the smallest bank (C) spends the most per step. With a cap of 3, Bank C stops after round 3, Bank B after round 10 and Bank A after round 16; final ε stays below the cap (2.97, 2.97, 2.69) and the model, starved of updates, ends at 0.490. The cap is enforced by the bank itself, before it trains, so no update ever exceeds the consortium's guarantee.

6.5 Leakage (scenario 3)

Direct leakage. A bank instructed to place transaction rows in its update (E9) is refused at the allow-list in the same round, logged as `leakage_blocked` and quarantined; nothing is stored. In the distributed run, the residency logs of the three banks show 0 raw rows sent, against 60,000 rows kept locally; Bank A sent 448 kB in total, all model updates, scores and counts.

Indirect leakage. We ran a loss-threshold membership-inference attack [17]: half of each bank's training rows were used for training and half held out from the same period; the attacker ranks rows by the final model's loss. An AUC of 0.5 means the attacker cannot tell members from non-members.

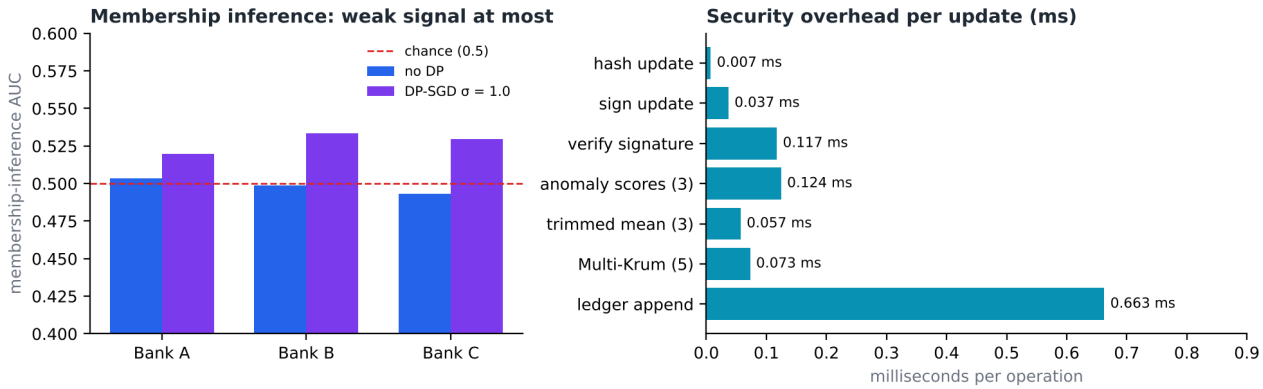


Figure 10. Left: membership-inference AUC per bank, without and with DP. Right: cost of the security operations per update.

Without DP the attack AUC is 0.504, 0.499, 0.493 for banks A, B and C (class-balanced 0.493, 0.503, 0.494): this small model does not memorise individual rows enough for the attack to work. With DP-SGD (σ = 1.0; ε ≈ 13 in this run, because each bank trains on half its rows) the AUC is 0.520, 0.533, 0.530 (class-balanced 0.499, 0.492, 0.528), a weak signal of at most 0.53. Both results are consistent with the DP guarantee, which at this ε would allow a much stronger attack, but at this scale the experiment does not show DP reducing empirical leakage: the model leaks little either way. The value of DP here is the formal, attacker-independent bound; stronger attacks (shadow models, gradient inversion on individual updates [16]) are needed to measure its practical effect, and are listed as future work.

6.6 Audit reconstruction after an incident (scenario 4)

Check	Result
Ledger after the 15-round attack run	75 records; all 75 records verified
Rounds reconstructed from the ledger alone	15; Merkle root recomputed from accepted update hashes matches the round commit and the on-chain anchor in 15 of 15 rounds
Time to reconstruct the 15 rounds	1.1 ms
Incident timeline	#23 r4 red_team_exercise → #26 r4 update → #29 r5 quarantine → #32 r5 update

Check	Result
Contract state	Bank A: approved, Bank B: approved, Bank C: quarantined; 15 anchors in 23 blocks
Insider edit of record #26 (accepted → true)	Verify: **content does not match its hash (record edited)** at record 26
ARIA: “Did anyone poison the model?”	Yes. Bank C was caught in round 5 (record #29): anomalous update (norm z=104.626, cosine=0.474). Current status of Bank C: quarantined.

Table 15. Audit reconstruction on the distributed system.

An auditor who receives only the ledger file and the contract state can rebuild who contributed to each round, which update was excluded and why, and when the quarantine decision was taken, and can prove that this history matches what was anchored at the time. An insider who edits a record to hide the exclusion is detected at the exact record, because the record no longer matches its signed hash.

6.7 Overhead

Security costs are negligible against training: per update, hashing takes 0.0075 ms, signing 0.0369 ms and verification 0.1174 ms; anomaly scoring for three banks 0.1245 ms; a ledger append 0.6626 ms. An update is 1,377 parameters (11.0 kB in float64, about 29 kB as JSON over HTTP). The real cost is privacy: a reference-engine round takes 0.039 s without DP and 3.404 s with per-sample gradients (about 90 times more).

7. Critical analysis

7.1 What the results show

- **Collaboration is worth it.** Every bank gains from the federation, and the federated model matches the centralised one. The requirement R6 is met.
- **Integrity is verifiable.** Every attack we injected was detected and attributed to the right bank, and every round can be proven after the fact (R2, R3, R5).
- **Residency is demonstrable.** The protocol cannot carry raw data, and banks hold evidence of what they sent (R1).
- **Empirical leakage is low.** A membership-inference attack achieves at most AUC 0.53, with or without DP.
- **Privacy is quantified but expensive.** A strong guarantee ($\varepsilon \approx 3$) costs about 30% of PR-AUC on this rare-event task (R4 met; utility only partly).

7.2 Performance and data

Our data is synthetic, and the generator's patterns are learnable; absolute PR-AUC values would be lower on real data, where laundering is rarer (AMLworld HI-Small has about 0.1%) and patterns adapt to controls. Seed variance (±0.03–0.04) is of the same order as some differences we discuss, so only differences larger than that should be read as real. A transaction-level MLP ignores the graph structure that laundering exploits; graph models would likely detect cross-bank cycles better but are harder to federate, since edges cross bank boundaries.

7.3 Security

Our detector relies on an honest majority and on attacks that change the size or direction of an update. With three banks, two colluding attackers would become the 'median'. Adaptive attacks that stay just below the z-score threshold [15] would be slowed by clipping and the robust rule but not identified. The aggregator sees individual updates, which is what makes detection possible but also means the operator must be trusted not to attack them; DP bounds what it could learn. Secure aggregation would remove that visibility, which is the fundamental trade-off between confidentiality of updates and accountability of participants. Transport is plain HTTP inside a private network in the prototype; production needs mutual TLS, and keys should live in hardware security modules.

7.4 Privacy and regulation

DP protects the training data of each bank against what can be inferred from its updates and from the final model, under the stated ε. It does not make the model anonymous in the GDPR sense automatically; that assessment belongs in a data protection impact assessment. The alert minimisation (counts only) respects the tipping-off prohibition, but even counts per round could reveal that a bank is handling a burst of suspicious activity; aggregating counts over longer windows would reduce this. Cross-border transfers inside the EEA are not restricted by Chapter V GDPR; a consortium including a UK or Swiss bank would need an adequacy basis.

7.5 Auditability

The ledger proves integrity of what was recorded, and the anchors prove when; neither proves that a bank trained honestly on real data. The operator signs the ledger, so a compromised operator key could produce a consistent but false history from that point on; on-chain anchoring limits this, because anchored roots cannot be rewritten. Our contract runs on a simulated chain by default and on a single-node development chain; a real deployment would run it on a permissioned network (for example Hyperledger Fabric) operated by the banks and the supervisor, so no single party controls it.

7.6 Engineering limits

The Flower + PyTorch + Opacus engine was implemented and its glue (client, strategy, guard, contract) tested with stand-ins in the build environment, where PyTorch and Flower could not be installed; the reported numbers therefore come from the reference engine and the distributed system. Running the Flower simulation on the demonstration laptop is part of defence preparation. Federated evaluation weights bank scores by their number of illicit test cases, an approximation of pooled PR-AUC.

7.7 Improvement paths

Area	Improvement	Expected effect
Data	Re-run the matrix on IBM AMLworld (HI-Small, LI-Small)	Realistic absolute numbers
Model	Federated graph neural networks with privacy-preserving cross-bank edges	Detect cross-bank cycles and fan-in/out
Privacy	Tune DP (larger batches, fewer rounds, adaptive clipping); user-level DP across rounds	Better utility at $\varepsilon \leq 6$
Security	Mutual TLS, HSM-held keys; secure aggregation combined with verifiable norm bounds	Hide updates while keeping poisoning resistance
Robustness	Adaptive and colluding attacks; reputation over time; larger consortia	Robustness beyond an honest majority of 3
Governance	Contract on Hyperledger Fabric with bank and supervisor peers; multi-signature approval	No single point of trust
Operations	Kubernetes deployment, alert routing to case management, DPIA template	Path to a pilot

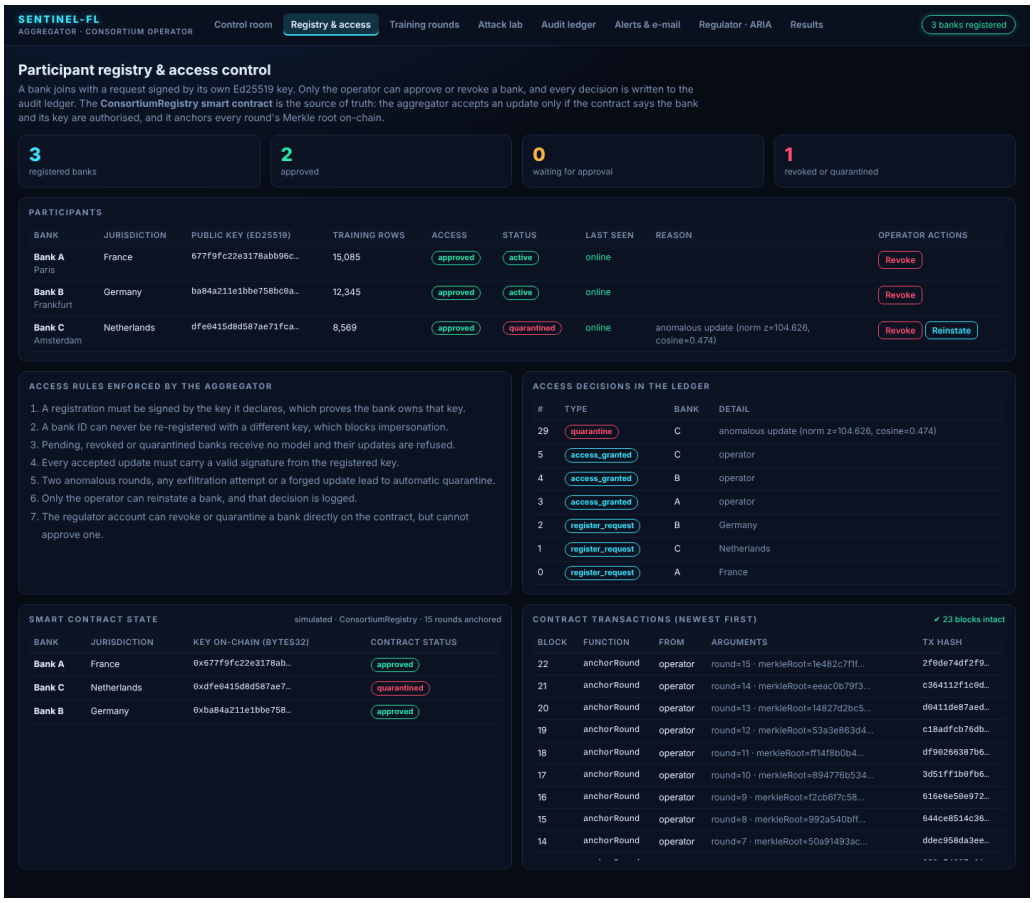
Table 16. Improvement paths.

8. Conclusion

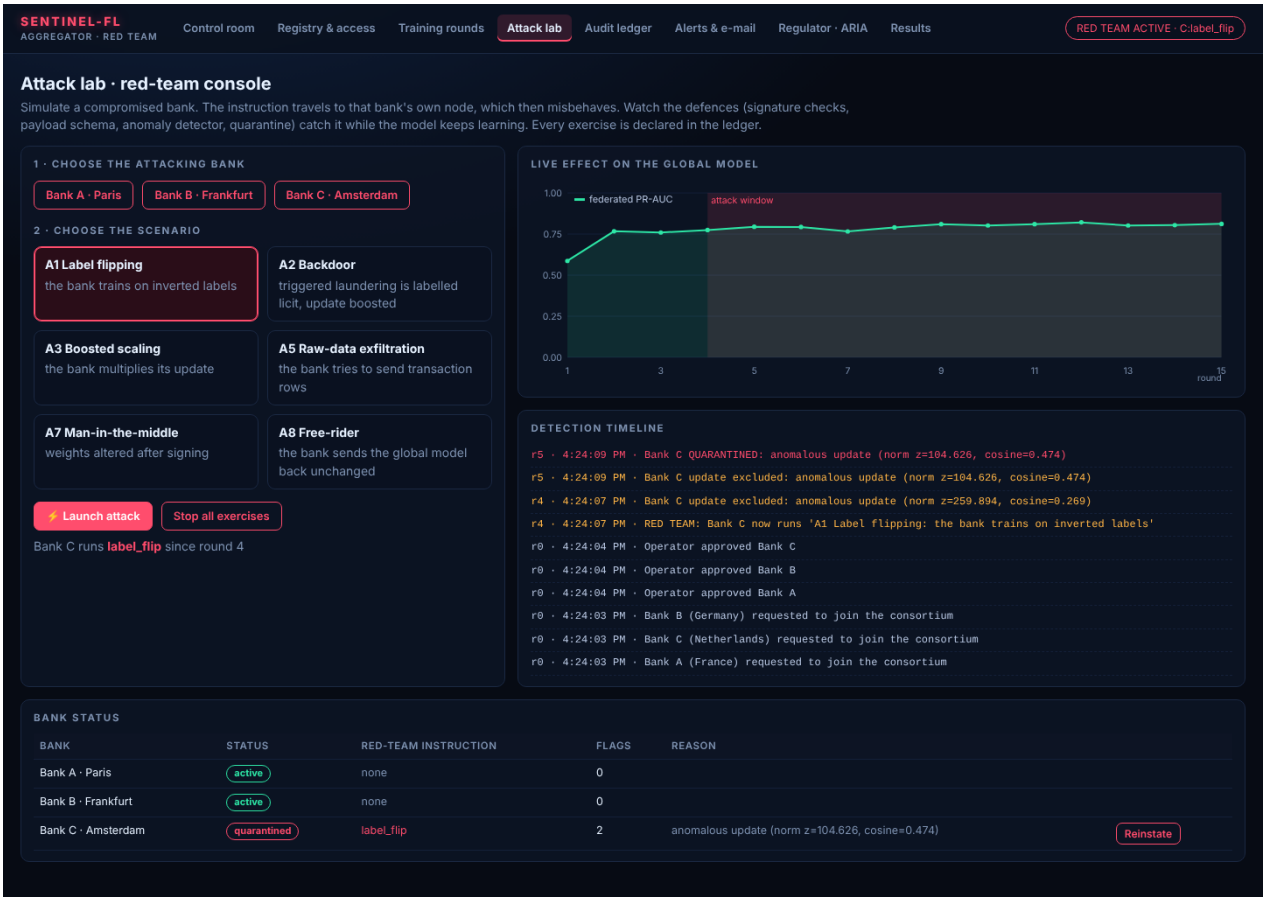
SENTINEL-FL shows that a consortium of banks can train an AML detection model together without moving transaction data, while giving a supervisor verifiable evidence of who took part, what each participant contributed, that a poisoning attempt was detected and handled, and that the record of all this was not rewritten. The federated model matches a centralised one on our data; defences restore an attacked model to its honest level and identify the attacker; differential privacy gives a measurable guarantee at a measurable cost. The main open problems are utility under strong privacy, robustness beyond an honest majority, and validation on real or AMLworld data, which define the next steps.

Annex A. Screenshots of the running system

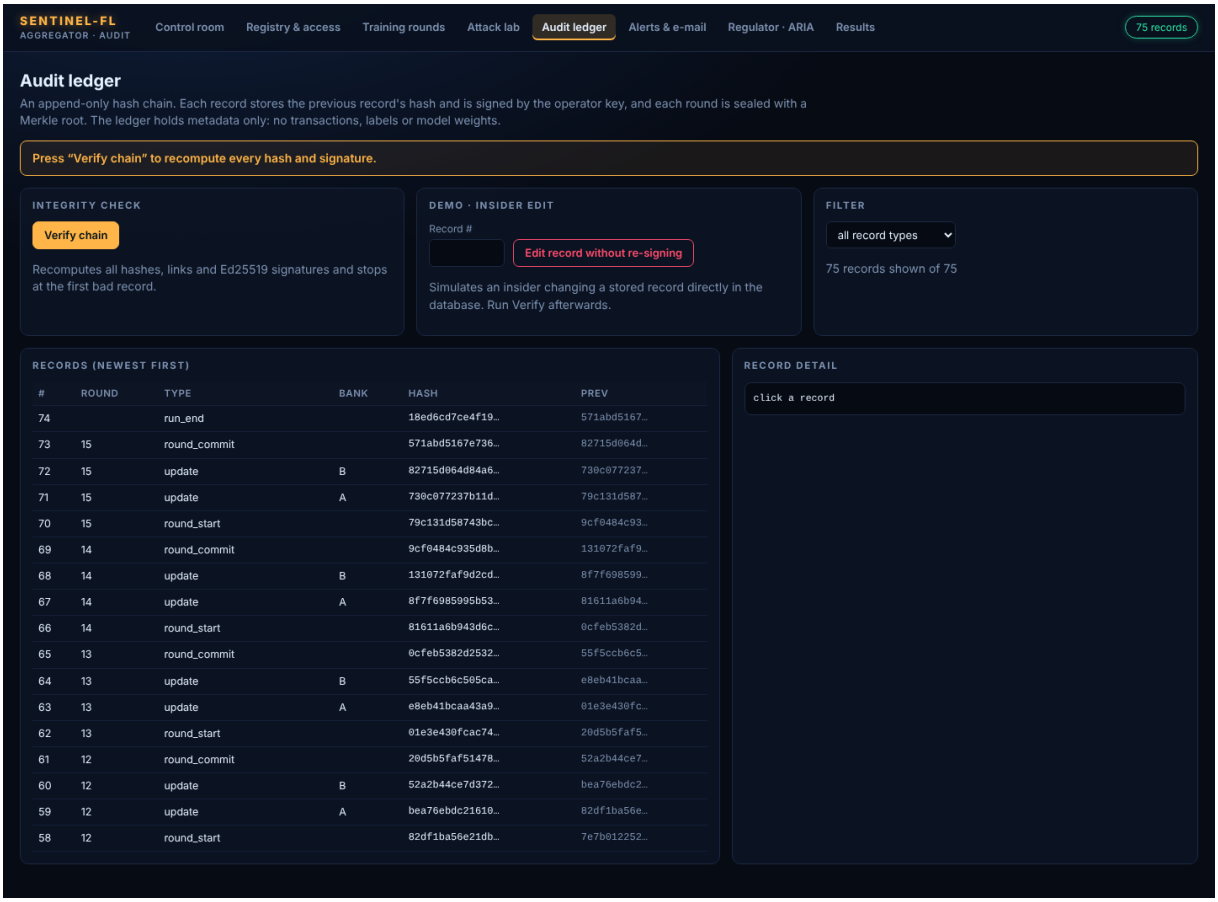
Screenshots from a live 15-round run on one laptop: aggregator, Banks A–C as separate processes, Bank C attacking from round 4.



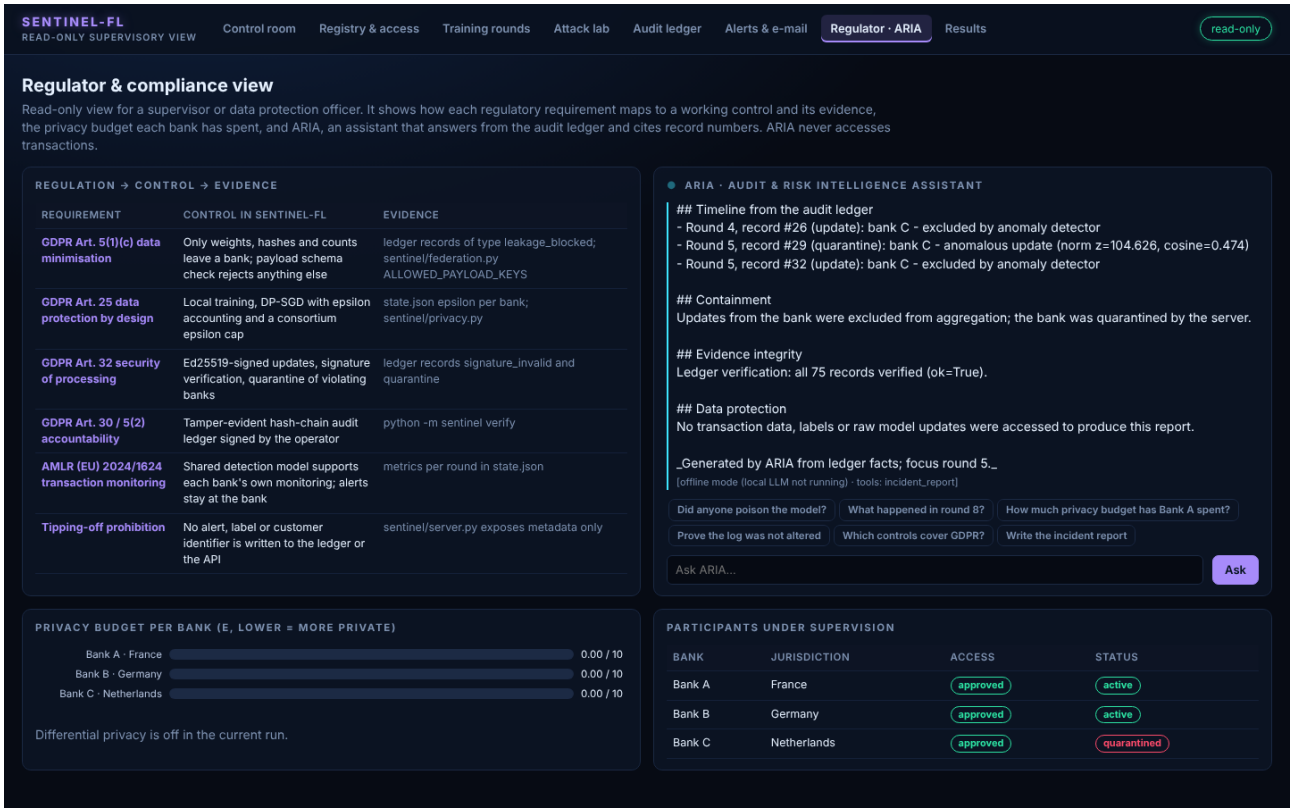
Registry & access: participants, ledger decisions, smart-contract state and transactions (anchorRound per round).



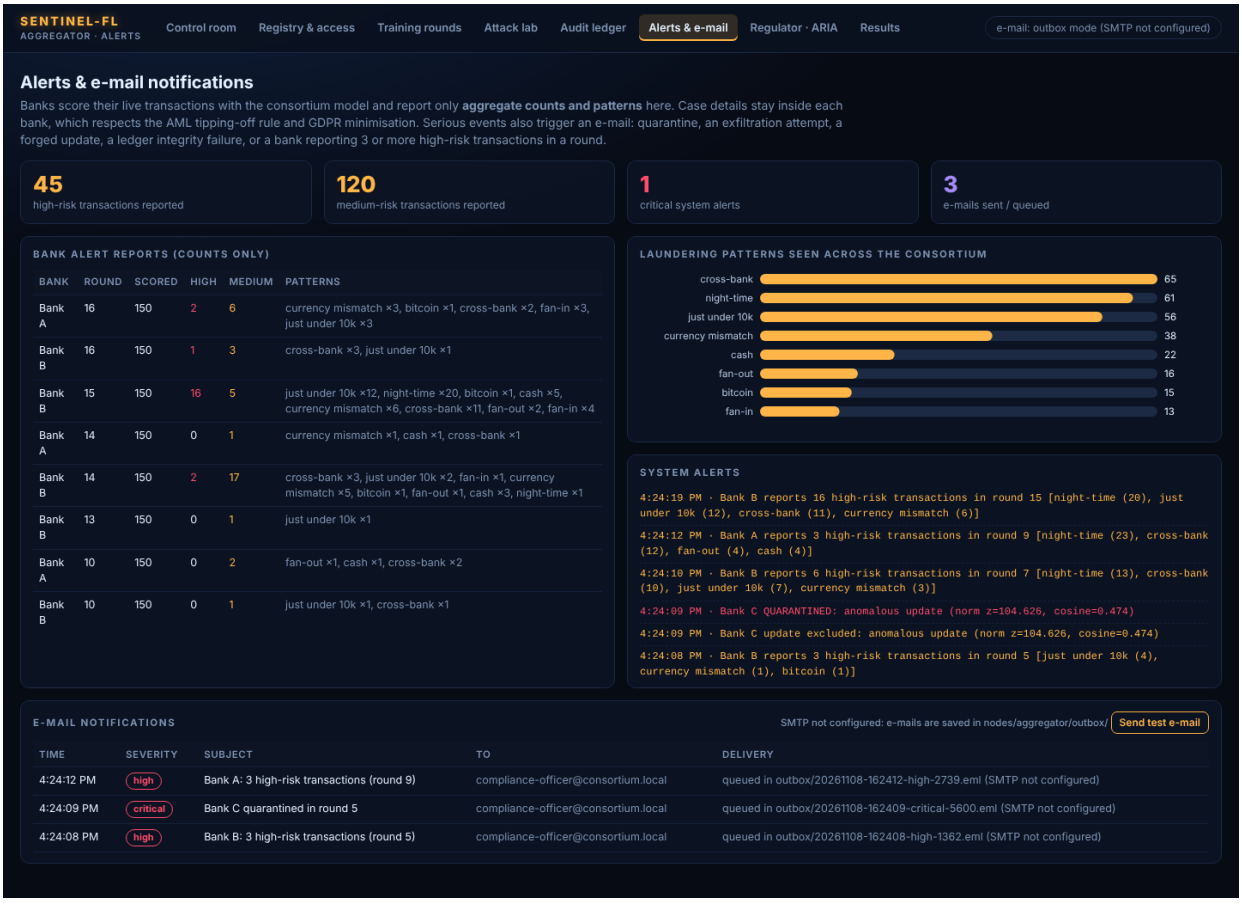
Attack lab: scenario chosen live, detection timeline, bank status.



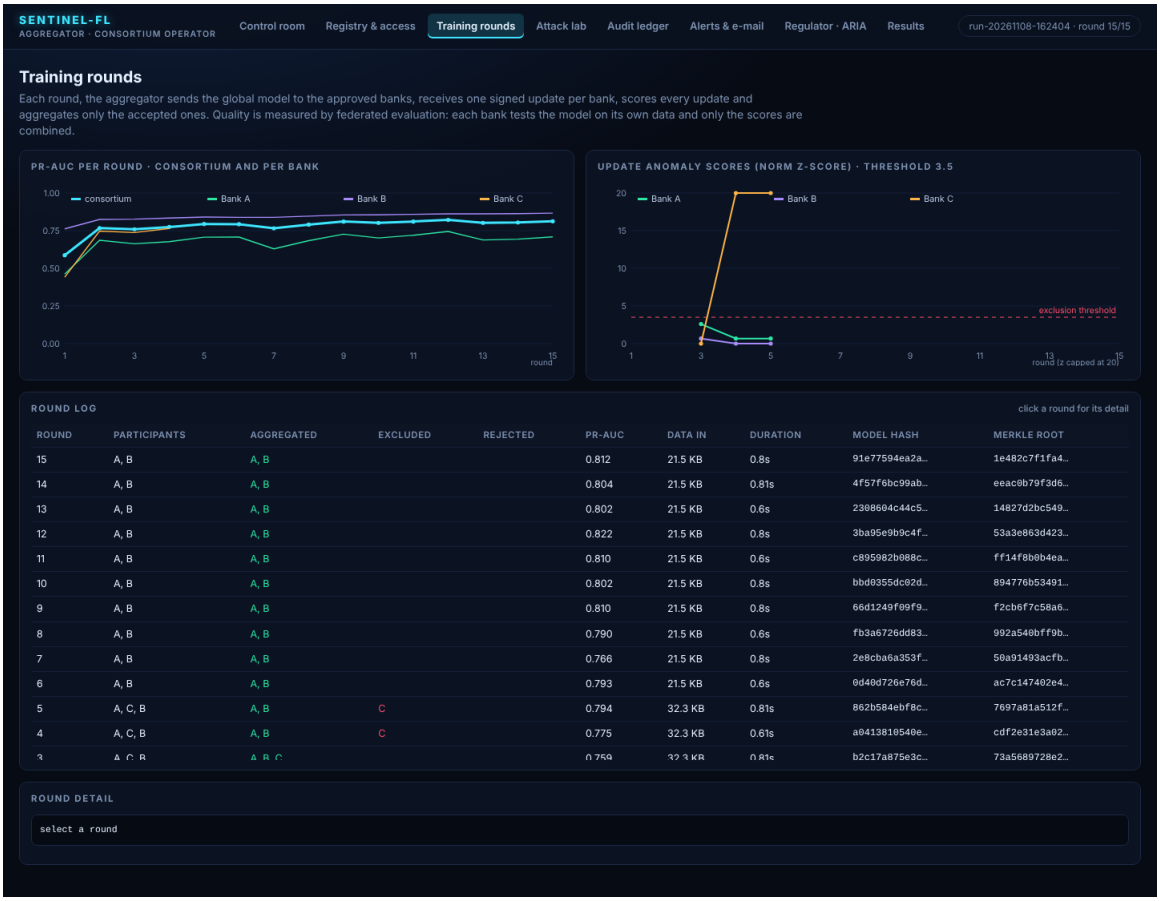
Audit ledger: signed records, verification, insider-edit test.



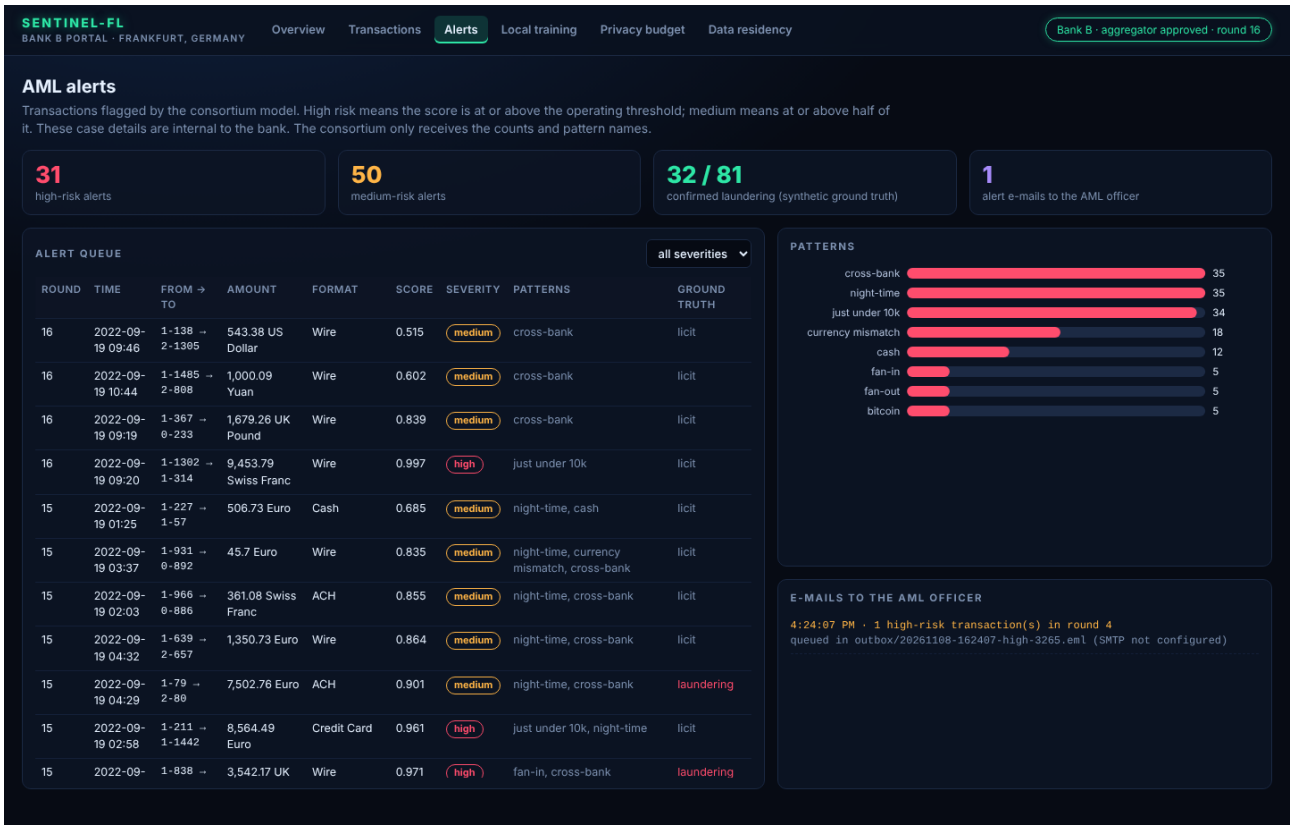
Regulator view: regulation → control → evidence, privacy budgets, ARIA incident report with ledger citations.



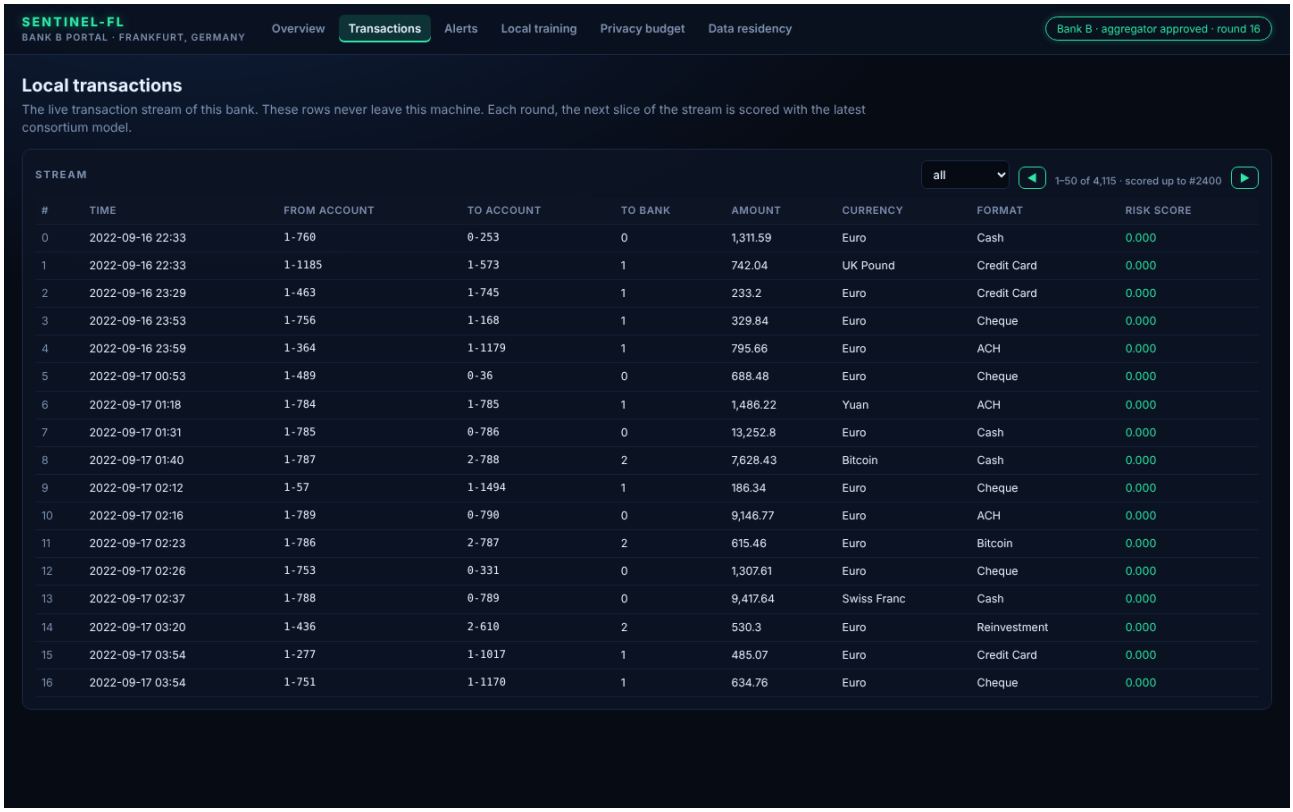
Alerts & e-mail: bank alert counts and patterns, system alerts, e-mail log.



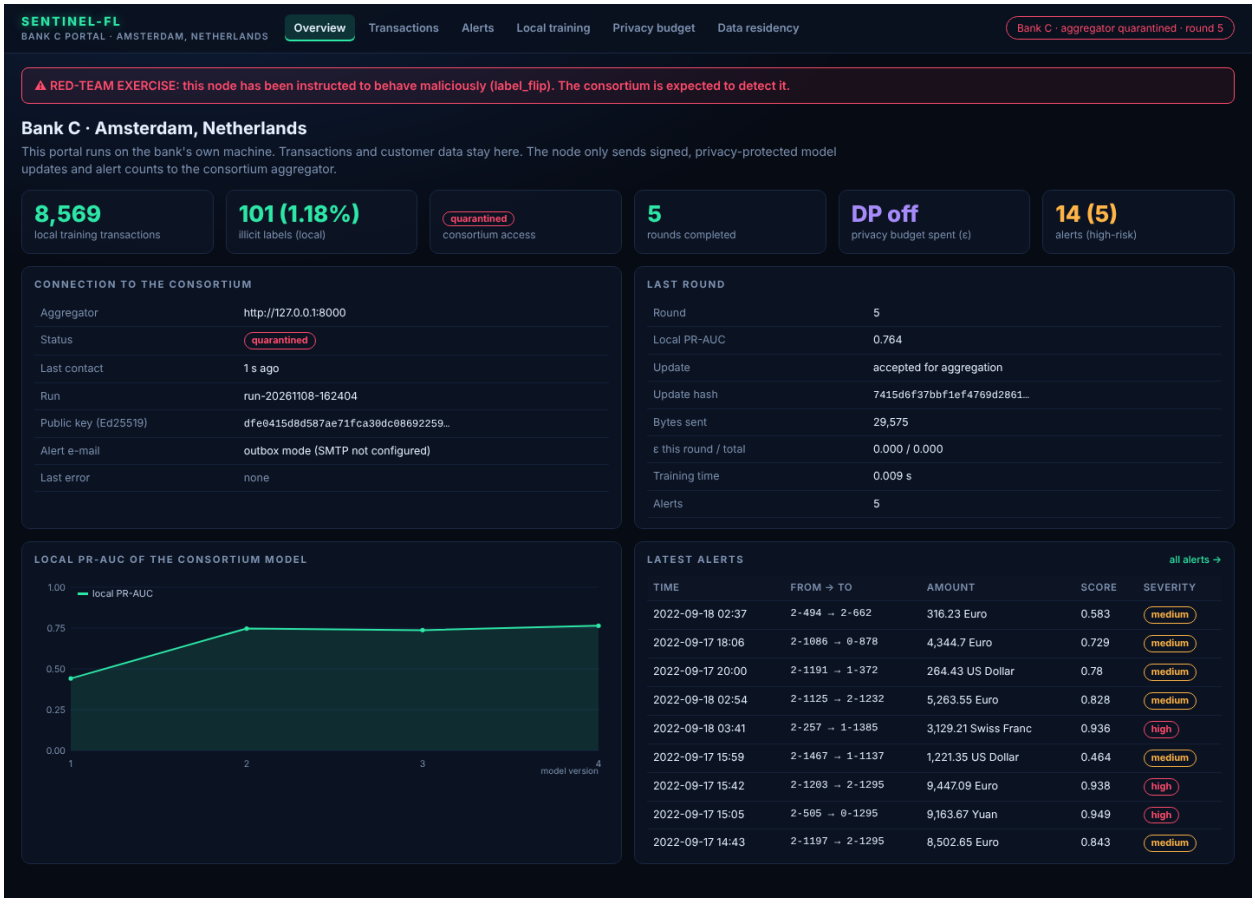
Training rounds: PR-AUC per bank, anomaly z-scores, round log with model hash and Merkle root.



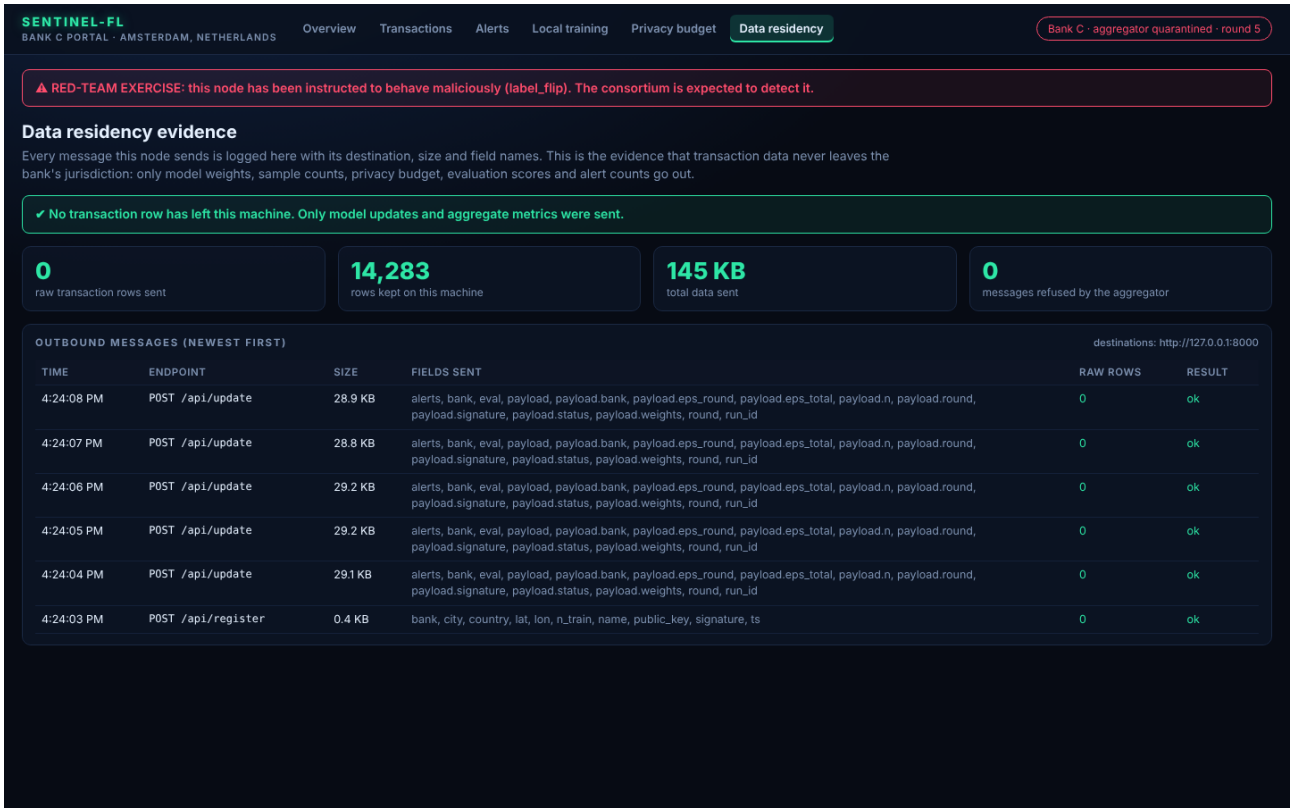
Bank B portal, AML alerts: case details stay in the bank.



Bank B portal, transactions: the private live stream with risk scores.



Bank C portal during the red-team exercise: quarantined by the consortium.



Bank C portal, data residency: every outbound message logged, 0 raw rows sent.

Annex B. Traceability to the brief

Brief item	Where it is met
Phase 1: scoping note, assets, threats, problem	Section 2; Phases 1–3 document
Phase 2: state of the art, comparative table, justification	Section 3, Tables 5–7
Phase 3: architecture diagrams, functional and technical description, attack scenarios	Section 4, Figures 2–3, Table 9
Flower orchestration with ≥ 3 simulated bank nodes	sentinel_flower (ServerApp/ClientApp, 3 SuperNodes); distributed nodes
Differential privacy for model updates (Opacus)	Opacus PrivacyEngine in sentinel_flower/task.py; DP-SGD + RDP in sentinel/privacy.py
Smart contracts for participant registration and access control	contracts/ConsortiumRegistry.sol; registry_chain.py; Registry page
Optional blockchain audit log	Signed hash-chain ledger with Merkle roots anchored in the contract
Dashboard for regulators and compliance officers	Regulator · ARIA page, Alerts page, 14 pages in total
Phase 4: documented code, configurations, test environment, journal	Section 5, Tables 10–13, repository docs/
Python, PyTorch, Flower; Docker, Kubernetes; Prometheus, Grafana	All except Kubernetes (Docker Compose used; Kubernetes is an improvement path)
Phase 5: five test scenarios, logs, tables, graphs, critical analysis	Sections 6–7, experiments/
Phase 6: report, defence, demonstration, conference proposal	This report; defence slides; live demo; conference proposal
Weekly reports	Weeks 1, 2, 3, 4–5, 6–7 sent to the referent

Annex C. Team contributions

Member	Role	Main contributions
Vijay Dipak Bhosale	Project Manager, Regulatory & Risk Lead	Scope and regulatory analysis, governance model, traceability, MS Project plan and risk register, weekly reports, integration, final report, defence coordination
Abdul Fathah Chappan Puthiya Purayil	FL & AI Lead	ML task and dataset design, model, FedAvg/FedProx, Flower port, baselines and FL experiments, federated evaluation
Mereena Mariam Jibu	Cybersecurity & Privacy Lead	Threat model, DP-SGD and Opacus, ε accounting, attack scenarios A1–A8, defences, membership-inference and leakage tests
Niranjan Satheesh Thandassery	Architecture & Auditability Lead	Architecture diagrams, ledger and Merkle anchoring, smart-contract registry, Docker and monitoring, dashboards and audit tests

Annex D. Reproducing the results

```
python -m unittest discover -s tests   # 21 tests
python -m sentinel matrix   # experiments E1–E10 (≈ 4 min)
python scripts/phase5.py ; python scripts/phase5_mia.py   # seeds, ε, overhead, audit, membership inference
python -m sentinel showcase   # one-laptop live demo (start_demo.bat)
python -m sentinel_flower.simulate --attack label_flip   # Flower + PyTorch + Opacus
```

References

[1] McMahan, B. et al. (2017). Communication-efficient learning of deep networks from decentralized data. AISTATS.

[2] Li, T. et al. (2020). Federated optimization in heterogeneous networks. MLSys.

[3] Kairouz, P. et al. (2021). Advances and open problems in federated learning. Foundations and Trends in Machine Learning 14(1–2).

[4] Beutel, D. J. et al. (2020). Flower: a friendly federated learning research framework. arXiv:2007.14390.

[5] He, C. et al. (2020). FedML: a research library and benchmark for federated machine learning. arXiv:2007.13518.

[6] Roth, H. R. et al. (2022). NVIDIA FLARE: federated learning from simulation to real-world. arXiv:2210.13291.

[7] Abadi, M. et al. (2016). Deep learning with differential privacy. ACM CCS.

[8] Mironov, I. (2017). Rényi differential privacy. IEEE CSF.

[9] Yousefpour, A. et al. (2021). Opacus: user-friendly differential privacy library in PyTorch. arXiv:2109.12298.

[10] Bonawitz, K. et al. (2017). Practical secure aggregation for privacy-preserving machine learning. ACM CCS.

[11] Gentry, C. (2009). Fully homomorphic encryption using ideal lattices. ACM STOC.

[12] Blanchard, P. et al. (2017). Machine learning with adversaries: Byzantine tolerant gradient descent. NeurIPS.

[13] Yin, D. et al. (2018). Byzantine-robust distributed learning: towards optimal statistical rates. ICML.

[14] Bagdasaryan, E. et al. (2020). How to backdoor federated learning. AISTATS.

[15] Fang, M. et al. (2020). Local model poisoning attacks to Byzantine-robust federated learning. USENIX Security.

[16] Zhu, L., Liu, Z., Han, S. (2019). Deep leakage from gradients. NeurIPS.

[17] Shokri, R. et al. (2017). Membership inference attacks against machine learning models. IEEE S&P.

[18] Altman, E. et al. (2023). Realistic synthetic financial transactions for anti-money laundering models. NeurIPS Datasets and Benchmarks.

[19] Weber, M. et al. (2019). Anti-money laundering in Bitcoin: experimenting with graph convolutional networks for financial forensics. KDD Workshop on Anomaly Detection in Finance.

[20] Kim, H. et al. (2020). Blockchained on-device federated learning. IEEE Communications Letters 24(6).

[21] Regulation (EU) 2016/679 (General Data Protection Regulation).

[22] Directive (EU) 2015/849 (4th AML Directive); Regulation (EU) 2024/1624 (AMLR); Regulation (EU) 2024/1620 (AMLA).

[23] Regulation (EU) 2022/2554 (Digital Operational Resilience Act).

[24] Regulation (EU) 2024/1689 (Artificial Intelligence Act).

[25] Balle, B. et al. (2020). Hypothesis testing interpretations and Rényi differential privacy. AISTATS.

[26] Androulaki, E. et al. (2018). Hyperledger Fabric: a distributed operating system for permissioned blockchains. EuroSys.

[27] Hamrioui, S. (2026). Applied AI projects, Master 1 MSc, ESAIP: Projet Applicatif 3 brief.

[28] Suzumura, T. et al. (2019). Towards federated graph learning for collaborative financial crimes detection. arXiv:1909.12946.